



TALLER DE ESTILOS ARQUITECTÓNICOS Y STACK TECNOLÓGICO: DOCUMENTO TÉCNICO

ARQUITECTURA DE SOFTWARE

PRESENTADO POR:

**TOMÁS OSPINA ULLOA
DAVID SANTIAGO RODRÍGUEZ PRIETO
GUSTAVO SALVADOR AGUILAR**

PRESENTADO A:

ANDRÉS ARMANDO SÁNCHEZ MARTÍN

PONTIFICIA UNIVERSIDAD JAVERIANA

FACULTAD DE INGENIERÍA

INGENIERÍA DE SISTEMAS

29 de septiembre de 2026

TABLA DE CONTENIDO

I. INVESTIGACIÓN	6
1.1. Estilo Arquitectónico Hexagonal	7
<i>1.1.1. Definición</i>	7
<i>1.1.2. Clasificación del Estilo</i>	7
<i>1.1.3. Características</i>	7
<i>1.1.4. Historia y Evolución</i>	7
<i>1.1.5. Ventajas y Desventajas</i>	7
<i>1.1.6. Problemas y Patrones</i>	7
<i>1.1.7. Patrones Aplicables</i>	7
<i>1.1.8. Casos de Uso</i>	7
<i>1.1.9. Casos de Aplicación</i>	7
1.2. MongoDB	7
1.2.1. Definición	7
<i>1.2.2. Características Principales</i>	8
<i>1.2.3. Historia y Evolución</i>	8
<i>1.2.4. Ventajas y Desventajas</i>	9
<i>1.2.5. Casos de Uso</i>	9
<i>1.2.6. Casos de Aplicación</i>	10
1.3. Docker	11

	3
<i>1.3.1. Definición</i>	11
<i>1.3.2. Características Principales</i>	11
<i>1.3.3. Historia y Evolución</i>	11
<i>1.3.4. Ventajas y Desventajas</i>	12
<i>1.3.5. Casos de Uso</i>	13
<i>1.3.6. Casos de Aplicación</i>	13
1.4. Vue.js	14
<i>1.4.1. Definición</i>	14
<i>1.4.2. Características Principales</i>	14
<i>1.4.3. Historia y Evolución</i>	15
<i>1.4.4. Ventajas y Desventajas</i>	15
<i>1.4.5. Casos de Uso</i>	16
<i>1.4.6. Casos de Aplicación</i>	16
• GitLab usa Vue como framework principal de su frontend, con guías oficiales de desarrollo publicadas en su documentación (GitLab, s.f.).	17
1.5. REST y JSON:API	17
<i>1.5.1. Definición</i>	17
<i>1.5.2. Características Principales</i>	18
<i>1.5.3. Historia y Evolución</i>	18

	4
<i>1.5.4. Ventajas y Desventajas</i>	19
<i>1.5.5. Casos de Uso</i>	19
<i>1.5.6. Casos de Aplicación</i>	20
1.6. FastAPI	20
1.6.1. Definición	20
1.6.2. Características Principales	20
1.6.3. Historia y Evolución	20
1.6.4. Ventajas y Desventajas	20
1.6.5. Casos de Uso	21
1.6.6. Casos de Aplicación	21
1.7: Relación entre el Estilo Hexagonal y las Tecnologías Seleccionadas	21
1.8. Qué Tan Común es el Stack	22
II. ANÁLISIS ARQUITECTÓNICO	23
2.1. Contextualización del Problema	23
2.2. Matriz de Tácticas vs. Estilo y Stack	25
2.3. Registros de Decisión Arquitectónica (ADRs)	26
<i>2.3.1. ADR-001: Casos de Uso Explícitos en la Capa de Dominio</i>	26
<i>2.3.2. ADR-002: Documento Embebido para la Ficha de Animal</i>	27
<i>2.3.3. ADR-003: JSON:API como Contrato de Integración</i>	27
<i>2.3.4. ADR-004: Interfaz Reactiva con Vue en vez de Recarga de Página</i>	28

2.3.5. <i>ADR-005: Políticas de Reinicio Automático en Docker Compose</i>	29
2.3.6. <i>ADR-006: Validación de Esquema en el Borde, Separada de las Reglas de Negocio</i>	29
2.3.7. <i>ADR-007: Puertos de Repositorio con Adaptador MongoDB</i>	30
2.3.8. <i>ADR-008: Contenerización de los Tres Servicios con Docker Compose</i>	30
2.3.9. <i>ADR-009: Adaptador en Memoria para Pruebas del Dominio</i>	31
2.3.10. <i>ADR-010: Auto-Cierre de Postulaciones para Prevenir Doble Adopción</i>	32
2.4. Matriz de Atributos de Calidad vs. Estilo Hexagonal	32
2.5. Matriz de Mercado Laboral vs. Estilo y Stack	34
2.6. Matriz de análisis de Principios vs estilo	36
III. DISEÑO DE EJEMPLO PRÁCTICO	36
3.1. Diagrama de Alto Nivel (HLD)	36
3.2. Diagrama de Contexto (Nivel 1)	36
3.3. Diagrama de Contenedores (Nivel 2)	37
3.4. Diagrama Dinámico	38
3.5. Diagrama de Despliegue	39
3.6. Diagrama de Componentes (Nivel 3)	39
3.7. Modelo de Datos	40
IV. LECCIONES APRENDIDAS	41
Referencias	42

I. INVESTIGACIÓN

Para empezar, se exponen el estilo arquitectónico y las tecnologías del stack tecnológico designados:

- **Estilo Arquitectónico:** Hexagonal
- **Frontend:** Vue.js/JS
- **Backend:** FastAPI/Python
- **Persistencia:** MongoDB/Document
- **Estrategia de Integración:** REST/JSON:API
- **Despliegue:** Docker

A continuación, se presenta la investigación de las tecnologías del estilo y stack designados:

1.1. Estilo Arquitectónico Hexagonal

1.1.1. Definición

La Arquitectura Hexagonal, conceptualizada por Cockburn (2005) como el patrón de Puertos y Adaptadores, constituye un estilo arquitectónico orientado al aislamiento de la lógica central del negocio frente a tecnologías y componentes externos, tales como sistemas de gestión de bases de datos, interfaces de usuario, marcos de trabajo web o servicios de terceros. Su principio operativo radica en el establecimiento de fronteras explícitas donde el dominio define las interfaces y contratos que la infraestructura debe satisfacer para establecer comunicación. A diferencia de las arquitecturas multicapa tradicionales, caracterizadas por una dependencia estricta hacia la capa de persistencia, este enfoque desacopla el núcleo conceptual de los detalles de implementación. Asimismo, no debe confundirse con un patrón de diseño a nivel de código orientado a la resolución de problemas locales, ni con un marco de trabajo o tecnología vinculada a un lenguaje de programación específico.

1.1.2. Clasificación del Estilo

Desde una perspectiva taxonómica, la Arquitectura Hexagonal se clasifica dentro de las Arquitecturas Centradas en el Dominio (Domain-Centric Architectures). Este paradigma comparte fundamentación conceptual con la Arquitectura Cebolla (Onion Architecture) y la Arquitectura Limpia (Clean Architecture), cuyo principio rector es la inversión de dependencias para asegurar la primacía e independencia del núcleo de negocio (Bass et al., 2021; Richards & Ford, 2020).

1.1.3. Características

- **Regla de Dependencia Estricta:** Las dependencias del código siempre apuntan hacia adentro; el núcleo del dominio no conoce ni importa nada de la infraestructura.
- **Puertos (Ports):** Son interfaces o contratos definidos por el dominio. Actúan como puntos de entrada (driving) o salida (driven) de la aplicación, dictando qué se puede hacer sin especificar el cómo.
- **Adaptadores (Adapters):** Son implementaciones concretas de la infraestructura que traducen tecnologías específicas (como HTTP en FastAPI o BSON en MongoDB) a llamadas que los puertos entienden.
- **Agnosticismo Tecnológico:** El framework web, la base de datos o el sistema de mensajería son tratados como "detalles" periféricos e intercambiables.
- **Testabilidad Aislada:** Permite probar el 100% de la lógica de aplicación utilizando adaptadores falsos (mocks o in-memory) sin levantar servidores ni bases de datos.

1.1.4. Historia y Evolución

- **2005:** Alistair Cockburn postula formalmente el patrón "Puertos y Adaptadores", popularizándolo bajo el nombre visual de "Arquitectura Hexagonal" para ilustrar que un sistema puede tener múltiples puntos de conexión.
- **2008:** Gana tracción en la comunidad de Diseño Guiado por el Dominio (DDD) como la estructura técnica ideal para proteger el modelo de dominio.
- **2012:** Jeffrey Palermo propone la Arquitectura Cebolla (Onion Architecture), formalizando la estructura de anillos concéntricos basada en los principios de Cockburn.
- **2015:** Robert C. Martin (Uncle Bob) publica "Clean Architecture", consolidando las ideas de Hexagonal y Cebolla en un estándar unificado y masivamente adoptado en la industria.
- **Actualidad:** Es un estándar de facto para construir microservicios robustos y sistemas empresariales donde la mantenibilidad a largo plazo y la independencia tecnológica son críticas.

1.1.5. Ventajas y Desventajas

- **Ventajas**
 - Independencia total de bases de datos, frameworks e interfaces de usuario.
 - Pruebas unitarias ultrarrápidas y completamente aisladas de la infraestructura externa.
 - Facilita postergar decisiones técnicas críticas (ej. comenzar el desarrollo sin elegir la base de datos definitiva).
 - Tolerancia extrema al cambio técnico; sustituir un proveedor o framework no afecta las reglas de negocio.

- **Desventajas**

- Genera una gran cantidad de código inicial (boilerplate) debido a la creación explícita de puertos, adaptadores y mapeadores.
- Curva de aprendizaje pronunciada para equipos acostumbrados a arquitecturas acopladas a la base de datos (MVC tradicional).
- Añade indirección en el código; el flujo de ejecución no es visualmente directo, obligando a navegar de la interfaz a su implementación.

1.1.6. Problemas y Patrones

Entre los principales problemas que aborda este estilo se encuentra el acoplamiento tecnológico fuerte, caracterizado por la mezcla del código de negocio con consultas a bases de datos o validaciones web, lo cual deriva en código de difícil mantenimiento. Asimismo, resuelve la imposibilidad de ejecutar pruebas automatizadas rápidas debido a la dependencia directa de bases de datos reales, servicios de terceros o conexiones de red. Finalmente, previene la «obsolescencia por marco de trabajo», evitando que un sistema completo deba reescribirse cuando una tecnología de infraestructura se vuelve obsoleta o cambia drásticamente de versión.

1.1.7. Patrones Aplicables

- **Inyección de Dependencias (DI):** Indispensable para inyectar los adaptadores concretos en los puertos sin que el dominio los instancie directamente.
- **Patrón Repository:** El puerto de salida por excelencia para abstraer la persistencia y lectura de colecciones de datos (ej. abstracción sobre MongoDB).

- **Data Transfer Objects (DTO):** Se usan en los adaptadores para transformar y validar estructuras de datos de la infraestructura (ej. Pydantic o JSON) antes de convertirlas a entidades puras del dominio.
- **Command Query Responsibility Segregation (CQRS):** Aplicado frecuentemente para separar flujos de lectura y escritura a través de puertos distintos.

1.1.8. Casos de Uso

Este estilo es adecuado para sistemas con reglas de negocio complejas y de larga vida útil, tales como plataformas de gestión con restricciones estrictas en transiciones de estado, microservicios, dominios altamente regulados y proyectos que requieren evolución tecnológica progresiva sin interrupción operativa. Por el contrario, su uso no se recomienda en aplicaciones tipo CRUD simples de flujo directo, prototipos de validación rápida (MVP), scripts de uso único o proyectos con plazos de comercialización sumamente restrictivos, donde el esfuerzo de abstracción inicial comprometería la entrega oportuna.

1.1.9. Casos de Aplicación

- **Netflix:** Documentó y expuso la adopción de la Arquitectura Hexagonal en sus servicios backend de streaming para aislar la lógica de core de las múltiples interfaces (APIs REST, GraphQL) y bases de datos transaccionales.
- **Patreon:** Utilizó principios hexagonales y tácticas de DDD para desacoplar su monolito histórico en servicios independientes, aislando reglas críticas de membresía y facturación de la infraestructura heredada.

- **Neo-bancos y Fintech (ej. NuBank, N26):** Aplican puertos y adaptadores de forma rigurosa para aislar los cálculos financieros y transacciones del núcleo del continuo cambio en las pasarelas de pago, regulaciones y plataformas móviles.

1.2. MongoDB

1.2.1. Definición

MongoDB es una base de datos NoSQL de código abierto, orientada a documentos, que almacena la información en formato BSON, una representación binaria de JSON, permitiendo estructuras de datos flexibles, anidadas y con esquema dinámico. (MongoDB, Inc., s.f.).

No obstante, no es una base de datos relacional ya que no trabaja con tablas, filas ni claves foráneas nativas, ni JOINS como en SQL. Asimismo, no impone un esquema fijo por diseño, aunque desde la versión 3.6 admite validación de esquema opcional vía JSON Schema, y no es una base de datos de grafos ni puramente clave-valor, sino específicamente orientada a documentos.

1.2.2. Características Principales

- Modelo de documentos BSON, que permite anidar arrays y subdocumentos dentro de un mismo registro.
- Esquema flexible (schema-less), con validación opcional cuando se necesita.
- Escalabilidad horizontal nativa mediante sharding.
- Alta disponibilidad mediante Replica Sets.
- Lenguaje de consultas propio (MQL) y un aggregation pipeline potente para transformaciones y reportes.

- Índices simples, compuestos, geoespaciales, de texto completo y TTL, de expiración automática de documentos.
- Transacciones ACID multi-documento desde la versión 4.0, en 2018.
- Drivers oficiales para prácticamente todos los lenguajes; en Python, PyMongo síncrono y Motor asíncrono (MongoDB, Inc., s.f.).

1.2.3. Historia y Evolución

- **2007:** Fundada como 10gen por Dwight Merriman, Eliot Horowitz y Kevin Ryan, veteranos de DoubleClick (MongoDB, Inc., s.f.).
- **2009:** MongoDB se libera como proyecto de código abierto independiente.
- **2013:** 10gen cambia su nombre a MongoDB, Inc. (MongoDB, Inc., s.f.).
- **2015:** Se adopta WiredTiger como motor de almacenamiento por defecto, mejorando concurrencia y compresión.
- **2017:** MongoDB Inc. sale a bolsa en el NASDAQ, bajo el símbolo MDB.
- **2018:** la versión 4.0 introduce transacciones ACID multi-documento.
- **2019 en adelante:** MongoDB Atlas, la base de datos como servicio en la nube, se convierte en el producto insignia de la compañía.

1.2.4. Ventajas y Desventajas

- **Ventajas**
 - Esquema flexible ideal para datos variables o jerárquicos.
 - Escalabilidad horizontal
 - Mapeo natural entre documento y JSON, lo que simplifica exponerlo vía APIs REST.

- Desarrollo ágil sin migraciones rígidas de esquema.
- Buen desempeño al leer un documento completo de una sola vez.

- **Desventajas**

- No es ideal para relaciones altamente normalizadas que requieren integridad referencial estricta, pues no existen foreign keys nativas.
- Los operadores \$lookup, equivalentes a un JOIN, son más costosos que un JOIN SQL optimizado.
- Tiende a la duplicación intencional de datos
- Mayor consumo de disco por el overhead de BSON
- Exige diseñar el esquema pensando en los patrones de consulta, no en la normalización clásica.

1.2.5. Casos de Uso

Entre los escenarios donde conviene usarla se encuentran: Primero, los catálogos con atributos variables entre ítems, como en un comercio electrónico donde una camiseta tiene talla y color mientras que un portátil tiene RAM y procesador. Segundo, los sistemas de gestión de contenido, donde un artículo, un video y una galería son tipos de contenido distintos que comparten poco entre sí. Tercero, los perfiles de usuario con campos dinámicos. Cuarto, los sistemas de IoT y logs de alta escritura, con millones de inserciones diarias y necesidad de expirar automáticamente los datos antiguos mediante índices TTL. Quinto, las aplicaciones con evolución rápida de esquema, como el producto mínimo viable de una startup. Por último, las fichas jerárquicas que varían según el tipo de entidad.

Por el contrario, entre los escenarios donde no conviene usarla se encuentran los sistemas contables o financieros, donde la integridad referencial estricta de un motor relacional evita

estados inconsistentes que en Mongo habría que validar manualmente en la aplicación. También se encuentran los reportes con muchas relaciones cruzadas, donde el optimizador de consultas SQL supera a encadenar varios \$lookup en Mongo. Finalmente se encuentran los esquemas estables y conocidos desde el diseño, como un registro civil, donde la validación estricta a nivel de base de datos es una ventaja y no una limitación.

1.2.6. Casos de Aplicación

- eBay usa MongoDB para catálogos de productos y metadata de búsqueda.
- Forbes migró su sistema de gestión de contenido a MongoDB por la flexibilidad de esquema.
- Toyota lo usa en sistemas de telemática de vehículos conectados.
- Genentech, en biotecnología, lo aplica en investigación genómica con datos de estructura variable.

1.3. Docker

1.3.1. Definición

Docker es una plataforma de contenedores que empaqueta una aplicación junto con todas sus dependencias en una unidad estandarizada y aislada (un contenedor) que comparte el kernel del sistema operativo anfitrión en lugar de virtualizarlo por completo.

Por el contrario, Docker no es una máquina virtual porque virtualiza hardware ni requiere un hipervisor o un sistema operativo completo por instancia. Tampoco es un orquestador de infraestructura, pues eso lo resuelven herramientas como Kubernetes o Docker Swarm y no es un lenguaje ni un framework de aplicación.

1.3.2. Características Principales

- Contenedores ligeros basados en namespaces y cgroups de Linux, que aíslan procesos, red y sistema de archivos sin virtualizar hardware.
- Imágenes inmutables, versionadas y construidas en capas a partir de un Dockerfile.
- Registries, como Docker Hub, para compartir y distribuir imágenes.
- Docker Compose, que permite definir y levantar aplicaciones multi-contenedor con un solo archivo YAML.
- Una imagen construida una vez corre igual en cualquier entorno con el motor de Docker instalado.
- Arranque en segundos, muy por debajo de los minutos que toma una máquina virtual tradicional.

1.3.3. Historia y Evolución

- **2013:** Solomon Hykes presenta Docker en la conferencia PyCon como proyecto interno de dotCloud, una plataforma PaaS (InfoQ, 2013).
- **2013:** Se libera como código abierto; dotCloud se renombra posteriormente a Docker, Inc. (InfoQ, 2013).
- **2014:** Docker desarrolla su propia librería, libcontainer, independizándose de LXC.
- **2015:** se funda la Open Container Initiative para estandarizar el formato de imágenes y contenedores entre proveedores.
- **2017:** mejoras en Docker Compose v3 e integración más estrecha con Kubernetes.
- **Actualidad:** Docker sigue siendo el estándar de facto en contenerización, incluso cuando se usan runtimes alternativos compatibles con OCI, como containerd o Podman.

1.3.4. Ventajas y Desventajas

- **Ventajas**

- Entornos reproducibles idénticos entre desarrollo, pruebas y producción.
- Arranque rápido y bajo overhead frente a una máquina virtual.
- Facilita pipelines de integración y despliegue continuo.
- Aísla dependencias entre servicios.
- Cuenta con un ecosistema amplio de imágenes preconstruidas.

- **Desventajas**

- Aislamiento más débil que una máquina virtual completa, pues comparte el kernel del host
- La persistencia de datos requiere volúmenes explícitos, ya que los contenedores son efímeros por diseño.
- La curva de aprendizaje crece cuando el sistema necesita orquestación y redes complejas.
- En Windows o Mac requiere una capa de virtualización adicional, lo que añade cierto overhead.

1.3.5. Casos de Uso

Conviene usarlo en arquitecturas de microservicios, como un comercio electrónico dividido en servicios de órdenes, pagos e inventario, cada uno desplegable de forma independiente. También en entornos de desarrollo reproducibles entre un equipo. Asimismo, puede usarse en pipelines de integración y despliegue continuo, donde se construye una imagen una sola vez y esa misma imagen se ejecuta en pruebas, en staging y en producción. A su vez, se usa en el despliegue de sistemas multi-servicio. Finalmente, en entornos efímeros de corta duración, como un ambiente de prueba distinto por cada rama de una funcionalidad.

No conviene usarlo, o no basta por sí solo, cuando se requiere aislamiento de seguridad extremo, pues los contenedores comparten el kernel del host y una máquina virtual completa ofrece mayor aislamiento. Tampoco conviene en aplicaciones monolíticas simples de un único servidor, sin necesidad de escalar ni portar a otros entornos, ni en sistemas embebidos con recursos extremadamente limitados, donde incluso el overhead mínimo del motor de contenedores resulta excesivo.

1.3.6. Casos de Aplicación

- Spotify despliega cientos de microservicios internos sobre contenedores Docker.
- PayPal migró gran parte de su infraestructura a contenedores para estandarizar despliegues.
- ADP, en nómina y recursos humanos, contenerizó su plataforma para acelerar los ciclos de despliegue.

1.4. Vue.js

1.4.1. Definición

Vue.js es un framework progresivo de JavaScript para construir interfaces de usuario basadas en componentes. Progresivo significa que se puede usar desde un solo widget dentro de una página existente hasta una aplicación de una sola página completa, con enrutador, gestión de estado y herramientas de construcción propias (Vue.js, s.f.).

Por el contrario, Vue no es un framework full-stack ni un backend al no tener acceso a base de datos ni lógica de servidor. Tampoco es un meta-framework de renderizado, pues por sí solo realiza renderizado en cliente, función que cumple Nuxt sobre Vue, no es una librería de componentes visuales, como Bootstrap o Vuetify, ya que Vue aporta el modelo de componentes,

mientras que el aspecto visual lo define el equipo y no impone una arquitectura de aplicación, pues no obliga a capas, puertos ni un patrón de estado concreto.

1.4.2. Características Principales

- Reactividad basada en proxies de JavaScript, a través de las funciones `ref`, `reactive` y `computed`, de modo que la vista se actualiza sola cuando cambia el estado.
- Componentes de archivo único (`.vue`), que combinan plantilla, lógica y estilos acotados en un mismo archivo.
- Composition API, introducida en Vue 3, que organiza la lógica reutilizable como funciones, con mejor soporte de tipos que la Options API clásica.
- Plantillas declarativas compiladas, donde el compilador optimiza la plantilla antes de llegar al Virtual DOM.
- Vue Router para rutas, Pinia para estado global, y Vite como herramienta de construcción y servidor de desarrollo.
- Curva de aprendizaje suave, con HTML y JavaScript estándar, sin requerir JSX ni TypeScript (Vue.js, s.f.; Vue School, 2025).

1.4.3. Historia y Evolución

- **2014:** Evan You, ex-ingeniero de Google que trabajaba con AngularJS, publica Vue.js, buscando extraer lo que le gustaba de Angular en algo más ligero (Vue.js, s.f.).
- **2016:** Se lanza Vue 2, con Virtual DOM, renderizado en servidor y adopción masiva, en especial en China y en el ecosistema de Laravel.
- **2020:** Se lanza Vue 3, con una reescritura en TypeScript, reactividad basada en Proxy y la Composition API; Vite reemplaza a Webpack y Vue CLI como herramienta de construcción recomendada, y Pinia reemplaza a Vuex como store oficial.

- **2023:** Finaliza el soporte de Vue 2.
- **2024:** Se lanza Vue 3.5, con optimizaciones de memoria y reactividad.
- **2026:** Vue 3.6 entra en fase de candidato a lanzamiento con Vapor Mode, una compilación opcional sin Virtual DOM (RepoJournal, 2026).

1.4.4. Ventajas y Desventajas

- **Ventajas**
 - Curva de aprendizaje baja, productivo en pocas horas.
 - Reactividad automática que reduce el código necesario para mantener la interfaz sincronizada con los datos.
 - Alta satisfacción de los desarrolladores, cercana al 84% según la compilación de la encuesta State of JS 2025. (tkrotoff, s.f.)
 - Bundle de JavaScript pequeño.
 - Herramientas oficiales integradas, como Vite, Router, Pinia y las DevTools.
- **Desventajas**
 - Menor demanda laboral que React, con una diferencia de uso reportada de alrededor de 17,6% frente a 44,7% en la encuesta de Stack Overflow de 2025. (tkrotoff, s.f.)
 - Menos librerías de terceros, sobre todo en componentes empresariales.
 - La migración de Vue 2 a Vue 3 fue costosa y dejó cierta fragmentación en tutoriales y paquetes.
 - La existencia de dos APIs, Options y Composition, obliga al equipo a fijar convenciones propias.

- No ofrece renderizado en servidor de fábrica, por lo que el posicionamiento en buscadores requiere Nuxt, lo que agrega complejidad.

1.4.5. Casos de Uso

Conviene usarlo al construir una aplicación de una sola página o un panel administrativo con mucho estado de interfaz que cambia, como formularios, tableros o listas filtrables. También conviene cuando el equipo es pequeño o tiene poca experiencia en frontend y necesita resultados rápidos y cuando se quiere modernizar por partes una aplicación existente, montando Vue en una sola sección.

No conviene usarlo, o conviene preferir otra opción, cuando el sitio es mayormente contenido estático y el posicionamiento en buscadores es crítico, donde Nuxt, Astro o un generador de sitios estáticos se ajustan mejor. Tampoco cuando la organización necesita el mayor grupo de contratación posible o un ecosistema empresarial muy específico, donde React o Angular tienen ventaja, o cuando la aplicación es móvil nativa, donde Flutter o React Native están más maduros.

1.4.6. Casos de Aplicación

- GitLab usa Vue como framework principal de su frontend, con guías oficiales de desarrollo publicadas en su documentación (GitLab, s.f.).
- Adobe modernizó el frontend de Behance y Portfolio con Vue (Monterail, 2026).
- Nintendo usa Vue en sitios regionales de Europa (Monterail, 2026).
- Xiaomi lo usa en sitios web y aplicaciones de alto tráfico (Monterail, 2026).
- El ecosistema de Laravel integra Vue de fábrica en sus starter kits, lo que lo volvió muy común en backends PHP (Monterail, 2026).

1.5. REST y JSON:API

1.5.1. Definición

REST, o Representational State Transfer, es un estilo arquitectónico para sistemas distribuidos definido por Roy Fielding en su tesis doctoral del año 2000 (Fielding, 2000). Se basa en recursos identificados por URIs, manipulados con los verbos estándar de HTTP, con comunicación sin estado entre cliente y servidor.

JSON:API es una especificación que define cómo un cliente solicita y modifica recursos, y cómo responde el servidor en JSON: estructura de documentos con datos, atributos, relaciones, recursos incluidos, errores y metadatos, además de filtros, paginación, ordenamiento e inclusión de relacionados, con un tipo de medio propio registrado ante la IANA (JSON:API, s.f.).

REST no es cualquier API que use JSON sobre HTTP: una API de tipo RPC con URLs como una acción de aprobar una postulación no es REST en sentido estricto. JSON:API, por su parte, no es un protocolo de transporte ni un reemplazo de HTTP, tampoco es GraphQL, pues el cliente no define la forma arbitraria de la respuesta sino que elige qué relaciones incluir y qué campos traer y no es OpenAPI o Swagger, ya que OpenAPI describe una API mientras que JSON:API estandariza el formato de sus mensajes, de modo que ambas se complementan.

1.5.2. Características Principales

- **REST** (Fielding, 2000).
 - Recursos con URI propia.
 - Interfaz uniforme, donde GET lee, POST crea, PATCH modifica parcialmente y DELETE elimina.
 - Ausencia de estado, pues cada petición lleva toda la información necesaria; códigos de estado HTTP con significado explícito.

- Arquitectura cacheable y en capas, donde intermediarios como proxies pueden participar.
- **JSON:API** (JSON:API, 2022)
 - Documento estructurado con datos, atributos y relaciones.
 - Documentos compuestos, que permiten incluir recursos relacionados en una sola petición mediante el parámetro `include`.
 - Filtros y paginación estandarizados; un objeto de error estándar con código, detalle y puntero de origen.
 - Negociación de contenido mediante el tipo de medio `application/vnd.api+json`.

1.5.3. Historia y Evolución

- **2000:** Roy Fielding define REST en su tesis doctoral en la Universidad de California, Irvine, como parte de su trabajo como coautor de la especificación HTTP/1.1 (Fielding, 2000).
- **2000-2010:** REST reemplaza progresivamente a SOAP y XML-RPC en las APIs web públicas.
- **2013:** Yehuda Katz, co-creador de Ember.js, redacta JSON:API a partir de lo aprendido con Ember Data; se registra el tipo de medio `application/vnd.api+json` ante la IANA.
- **2015:** Se publica la versión 1.0 final de JSON:API (JSON:API, s.f.).
- **2019:** JSON:API entra al núcleo de Drupal, en su versión 8.7, como módulo estable (Drupal, 2019).
- **2022:** Se publica JSON:API 1.1, con extensiones, perfiles e identificadores locales para crear recursos relacionados en una sola petición (JSON:API, 2022).

1.5.4. Ventajas y Desventajas

- **Ventajas de REST:** Es el estándar de facto, usado por el 93% de los equipos de API, entendido por cualquier cliente, herramienta o proxy; y aprovecha la infraestructura HTTP existente (Postman, 2025).
- **Ventajas de JSON:API:** Elimina la discusión recurrente sobre el formato, pues el equipo no debate cómo paginar ni cómo devolver errores; los documentos compuestos reducen los viajes de red; y los errores con puntero de origen facilitan mostrar validaciones en la interfaz.
- **Desventajas:** JSON:API es más verboso que un JSON plano, pues una respuesta simple tiene más anidación; es menos popular que REST sin convención estricta, y FastAPI no lo genera de fábrica, por lo que hay que serializar manualmente o con una librería adicional; las operaciones que no son de creación, lectura, actualización o eliminación, como aprobar una postulación, se modelan como un cambio de estado, lo cual es menos explícito que un endpoint de acción; y persiste cierto riesgo de sobre-obtención o sub-obtención de datos frente a GraphQL en pantallas muy compuestas.

1.5.5. Casos de Uso

Conviene usar REST junto con JSON:API cuando el dominio se modela naturalmente como recursos con relaciones, como animales, refugios y postulaciones. También cuando varios equipos o clientes consumen la misma API y conviene un contrato uniforme sin documentar convenciones propias y cuando se necesitan paginación, filtros y errores consistentes desde el principio.

No conviene usarlo cuando el cliente necesita consultas muy flexibles y heterogéneas por pantalla, donde GraphQL se ajusta mejor. Tampoco en comunicación interna entre

microservicios de alto rendimiento, donde gRPC ofrece mejor desempeño ni cuando se requieren actualizaciones en tiempo real del servidor al cliente, donde WebSockets o Server-Sent Events son más apropiados o en una API mínima de dos o tres endpoints, donde la verbosidad de JSON:API no se justifica frente a un REST simple.

1.5.6. Casos de Aplicación

- Drupal expone todo su contenido vía JSON:API desde su núcleo desde 2019, base de numerosos proyectos headless gubernamentales y de medios (Drupal, 2019).
- Ember Data usa JSON:API como formato por defecto; la especificación nació precisamente de ese proyecto.
- REST en general es el estilo de referencia de APIs públicas como las de GitHub, Stripe y Twilio.

1.6. FastAPI

1.6.1. Definición

FastAPI constituye un microframework web de alto rendimiento orientado al desarrollo de interfaces de programación de aplicaciones (API) en Python, cuya arquitectura se fundamenta de manera rigurosa en las anotaciones de tipos estáticas (type hints) del estándar del lenguaje (Ramírez, 2018). Asimismo, opera bajo la especificación de interfaz de pasarela de servidor asíncrona (ASGI), lo que le confiere un desempeño optimizado como enrutador asíncrono y ligero. En contraposición, no corresponde a un framework monolítico de tipo integral (como Django), debido a que no integra de forma nativa un mapeador objeto-relacional (ORM), sistemas de autenticación rígidos ni motores de plantillas para el renderizado del lado del servidor.

1.6.2. Características Principales

- Rendimiento extremadamente alto gracias a Starlette, comparable a ecosistemas como NodeJS y Go, al soportar operaciones asíncronas (async/await) de forma nativa.
- Validación, serialización y transformación de datos en tiempo de ejecución utilizando Pydantic y el tipado estático de Python.
- Generación automática e interactiva de documentación (Swagger UI y ReDoc) basada estrictamente en los estándares abiertos OpenAPI y JSON Schema.
- Sistema nativo de inyección de dependencias (Depends) diseñado para facilitar la modularidad, el manejo de conexiones a bases de datos y la abstracción de la seguridad.

1.6.3. Historia y Evolución

- 2015: Se aprueba el PEP 484, introduciendo el tipado estático opcional en Python, base fundamental tecnológica del futuro framework (Langa, 2015).
- 2018: Sebastián Ramírez inicia el desarrollo de FastAPI, motivado por la necesidad de combinar validación de datos, serialización y documentación automática sin acoplar múltiples librerías ineficientes en Flask (Ramírez, 2018).
- 2018: Se publica la primera versión de FastAPI como código abierto.
- 2019-2020: Experimenta una adopción masiva, convirtiéndose en uno de los repositorios de Python de más rápido crecimiento en GitHub gracias a su adopción en ciencia de datos y microservicios.
- Actualidad: Se consolida como un estándar de facto en la industria para construir microservicios rápidos y APIs RESTful en Python.

1.6.4. Ventajas y Desventajas

- **Ventajas:**

- Acelera drásticamente la velocidad de desarrollo al reducir errores humanos mediante el autocompletado nativo en los editores.
- Ofrece un rendimiento asíncrono superior para operaciones de entrada/salida (I/O).
- Mantiene la documentación de la API siempre actualizada sin esfuerzo adicional gracias a la autogeneración de OpenAPI.
- **Desventajas:**
 - El ecosistema de persistencia de datos está fragmentado, obligando al equipo a evaluar y acoplar librerías de terceros (como SQLAlchemy o Motor) al no existir un ORM oficial.
 - Su curva de aprendizaje puede ser engañosa, ya que un mal manejo de operaciones síncronas bloqueantes en rutas asíncronas puede degradar severamente el rendimiento.
 - Requiere escribir código adicional para unificar el manejo de excepciones del dominio hacia respuestas HTTP estándar.

1.6.5. Casos de Uso

El uso de FastAPI resulta especialmente adecuado en escenarios donde el dominio requiere el desarrollo de microservicios o la implementación de interfaces de programación de aplicaciones (API) REST de alto rendimiento. Asimismo, se beneficia sustancialmente en entornos caracterizados por cargas intensivas de entrada y salida (I/O), tales como el procesamiento simultáneo de peticiones concurrentes a bases de datos o servicios externos, y en

aquellos proyectos donde la interoperabilidad entre los equipos de desarrollo del lado del cliente (frontend) y del servidor (backend) se facilita mediante la generación automatizada de especificaciones OpenAPI OpenAPI. Adicionalmente, constituye una herramienta ampliamente extendida en la ingeniería de datos y el aprendizaje automático para la exposición y despliegue de modelos predictivos.

Por el contrario, su adopción no se recomienda en el desarrollo de aplicaciones monolíticas tradicionales orientadas a la renderización de HTML desde el servidor (Server-Side Rendering), ámbito en el cual marcos de trabajo integrales como Django ofrecen soluciones nativas de mayor madurez. Tampoco se aconseja en sistemas con un alto nivel de acoplamiento que requieran un panel de administración autogenerado desde las etapas iniciales del proyecto, ni en entornos donde el equipo de desarrollo no domine las implicaciones operativas del paradigma asíncrono en Python, lo que podría derivar en cuellos de botella en el rendimiento del sistema.

1.6.6. Casos de Aplicación

- Netflix utiliza FastAPI en el backend de Dispatch, su plataforma interna de código abierto para la gestión de incidentes y resolución de crisis (Netflix, 2020).
- Uber lo despliega en su infraestructura para soportar microservicios y exponer modelos de aprendizaje automático, aprovechando su velocidad y concurrencia.
- Microsoft lo ha incorporado en diversos equipos para construir servicios backend ágiles y cognitivos, integrándolo en despliegues en la nube y entornos Windows.
-

1.7: Relación entre el Estilo Hexagonal y las Tecnologías Seleccionadas

- **MongoDB y Hexagonal:** El estilo hexagonal exige que el dominio no conozca detalles de persistencia, por lo que MongoDB encaja de forma natural con esto porque el puerto

de repositorio se define en términos de las entidades del dominio, mientras que el adaptador oculta por completo los detalles de BSON y las consultas de Mongo. Como MongoDB no impone un esquema rígido, evolucionar el modelo del dominio no obliga a migraciones estructurales complejas del lado de la base de datos, lo que refuerza la idea central de Hexagonal de que la infraestructura es un detalle reemplazable y no el centro del diseño.

- **Docker y Hexagonal:** No existe una relación de patrón de diseño directa, pues Docker no forma parte del estilo arquitectónico en sí, pero sí una relación práctica importante. Así que, Hexagonal separa el dominio de los adaptadores y cada adaptador puede empaquetarse y desplegarse como un contenedor independiente, sin que un cambio en uno obligue a reconstruir los demás. Por ende, la separación lógica que propone Hexagonal se refleja aquí en una separación física de contenedores.
- **Vue.js y Hexagonal:** Hexagonal es un estilo del backend que aísla el dominio de la infraestructura. En este sistema, Vue ocupa el lugar de un actor primario que conduce la aplicación desde fuera del hexágono, a través del adaptador de entrada REST, y nunca conoce el dominio ni MongoDB, solo el contrato JSON:API. Además, el propio frontend aplica la misma idea a pequeña escala, haciendo que el módulo de acceso a la API actúe como puerto de la aplicación hacia el backend, detrás del cual hay dos adaptadores intercambiables con la misma interfaz, uno que llama a la API real y otro que sirve datos simulados en memoria respetando las mismas reglas de negocio, elegido por configuración de la misma forma en que el backend elige su repositorio.
- **REST/JSON:API y Hexagonal:** En Hexagonal, REST es exactamente un adaptador de entrada porque el enrutador de FastAPI traduce una petición JSON:API a una llamada a

un caso de uso, y traduce la respuesta de ese caso de uso de vuelta a JSON:API. A su vez, el dominio no sabe que existe HTTP, pues sus reglas lanzan excepciones de dominio propias, y solo el adaptador REST las traduce a códigos de estado con el formato de error de JSON:API. Si en el futuro se quisiera exponer los mismos casos de uso por GraphQL o por una interfaz de línea de comandos, bastaría con agregar otro adaptador de entrada sin tocar el dominio, que es la justificación central del estilo.

- **FastAPI y Hexagonal:** En Hexagonal, FastAPI es exactamente un adaptador de entrada porque su enrutador traduce una petición HTTP, previamente validada por Pydantic, a una llamada a un caso de uso, y traduce el resultado de ese caso de uso de vuelta a una respuesta HTTP. A su vez, el dominio no sabe que existe FastAPI, Pydantic o el protocolo web, pues opera exclusivamente con clases estándar de Python, y es únicamente el framework web el que, mediante su sistema de inyección de dependencias (Depends), se encarga de ensamblar y proveer los adaptadores de salida (como la base de datos) hacia el interior. Si en el futuro se quisiera cambiar de framework web por Django o exponer la misma lógica mediante tareas en segundo plano como Celery, bastaría con agregar otro adaptador de entrada sin tocar el dominio, que es la justificación central del estilo.

1.8. Qué Tan Común es el Stack

Cada pieza por separado es muy común en la industria: Docker es prácticamente universal en el despliegue moderno de software, MongoDB se mantiene consistentemente entre las bases de datos NoSQL más usadas a nivel mundial, FastAPI es uno de los frameworks de Python de más rápido crecimiento en adopción reciente (DZone, 2025); y Vue.js es el tercer

framework de frontend más usado globalmente, después de React y Angular (Stack Overflow, 2025).

Paralelamente, la combinación específica de FastAPI y MongoDB es habitual en APIs modernas orientadas a datos semi-estructurados y constituye una variante del conocido stack “FARM” (FastAPI, React, MongoDB), reemplazando React por Vue. A esta variante se le llamará en este caso “FAVM” (FastAPI, Vue, MongoDB).

Así pues, la evidencia real de adopción se sostiene mejor observando cada pieza por separado: la documentación oficial de FastAPI lista empresas que lo usan en producción, entre ellas Netflix, Uber, Microsoft y Cisco. Por otro lado, Vue.js es usado por GitLab, Adobe y Alibaba, entre otros y MongoDB cuenta con los casos de eBay, Forbes, Toyota y Genentech ya descritos. Consecuentemente, la combinación completa de las tres piezas se observa con más frecuencia en un tipo de proyecto específico: APIs de backend para modelos de aprendizaje automático, paneles administrativos internos y productos mínimos viables de software como servicio, que en nombres de grandes marcas asociados a esa combinación exacta.

A su vez, Hexagonal, a diferencia de las tecnologías anteriores, no es una decisión de nicho tecnológico sino una decisión de diseño transversal debido a que aparece en equipos que usan Python y FastAPI cuando el dominio de negocio es lo suficientemente complejo como para querer aislarlo de la infraestructura, sin importar si la base de datos es relacional o documental. En conjunto, no es el stack más popular del mercado, como MERN o Django con PostgreSQL, pero tampoco es exótico al ser una combinación real, documentada, con soporte maduro de librerías como Motor y Beanie, y con evidencia de uso en producción.

II. ANÁLISIS ARQUITECTÓNICO

2.1. Contextualización del Problema

El problema abarcado es un Sistema de Adopción de Mascotas, una plataforma web que conecta refugios de animales con posibles adoptantes, implementada bajo el estilo arquitectónico, este resuelve un problema operativo concreto de los refugios, el cual radica en la gestión y publicación de animales disponibles y las solicitudes de adopción de forma manual o dispersa entre distintos canales, lo que genera un riesgo de comprometerse con más de un adoptante por el mismo animal. La plataforma centraliza ese proceso y automatiza la regla de negocio más delicada: Al aprobar una postulación, todas las demás postulaciones pendientes sobre ese mismo animal se cierran automáticamente, sin depender de que un operador humano recuerde hacerlo. A raíz de ello, el dominio del sistema se modela alrededor de cuatro entidades interrelacionadas (Ver Diagrama de Contexto de la Sección 3 para mayor comprensión):

- **Refugio:** Organización que publica animales disponibles para adopción.
- **Animal:** Ficha con especie, edad, estado de salud, temperamento y fotos, cuyos campos varían según la especie (perro, gato, ave), puede estar en estado disponible, postulado o adoptado.
- **Adoptante:** Persona que consulta animales disponibles y postula a uno de ellos.
- **Postulación:** Vincula a un adoptante con un animal, y puede estar pendiente, aprobada o rechazada.

Sobre estas entidades operan tres reglas de negocio centrales: Primero, no se puede postular a un animal que no esté disponible. Segundo, un adoptante no puede tener más de una postulación pendiente sobre el mismo animal. Por último, al aprobar una postulación, el animal pasa a adoptado y todas las demás postulaciones pendientes sobre ese animal se rechazan de forma automática.

El flujo funcional de extremo a extremo que da cuerpo a este ejemplo es el siguiente: Primero, el refugio publica un animal disponible. Luego, el adoptante consulta su ficha y postula.

Posteriormente, el sistema valida que el animal siga disponible y que no exista ya una postulación duplicada. Por último, el refugio revisa la postulación recibida y decide aprobarla o rechazarla y, si la aprueba, el sistema marca el animal como adoptado y cierra automáticamente cualquier otra postulación pendiente sobre él. (Ver el Diagrama Dinámico de la Sección 3 para mayor comprensión)

Asimismo, este caso se elige porque combina una superficie de implementación acotada con solo cuatro entidades y una regla de negocio genuinamente no trivial (el cierre automático de postulaciones), lo suficientemente rica como para justificar por qué vale la pena aislar el dominio de la infraestructura mediante el estilo Hexagonal, en lugar de resolverlo como un CRUD simple. Paralelamente, FastAPI expone los casos de uso como API REST siguiendo la convención JSON:API, MongoDB persiste cada animal como un documento con su ficha de salud y fotos embebidas, dada la variabilidad de campos entre especies, Vue.js consume esa API para ofrecer tanto el catálogo y el formulario de postulación del adoptante como el panel de aprobación del refugio y Docker empaqueta los tres servicios para poder levantar el sistema completo con un solo comando.

2.2. Matriz de Tácticas vs. Estilo y Stack

Esta matriz recorre los nueve atributos de calidad según ISO/IEC 25010:2023, identificando una táctica arquitectónica real aplicada en el proyecto para cada uno, mostrando cómo la materializan el estilo Hexagonal y el stack tecnológico, derivando cada una en un ADR.

Tabla 1: Tácticas arquitectónicas aplicadas por atributo de calidad

Atributo	Táctica	Aporte del estilo Hexagonal	Aporte del stack	ADR
----------	---------	-----------------------------	------------------	-----

Adecuación funcional	Encapsular las reglas de negocio como casos de uso explícitos	Los casos de uso del dominio son el único lugar donde vive la regla completa	FastAPI solo orquesta la llamada al caso de uso; Pydantic no contiene lógica de negocio	ADR-001
Eficiencia de desempeño	Gestionar la demanda de recursos evitando JOINS innecesarios	El adaptador decide cómo se consulta el dato; el dominio solo pide el animal completo	MongoDB embebe fotos, ficha de salud y etiquetas en un solo documento	ADR-002
Compatibilidad	Adoptar un protocolo de integración estándar	El adaptador REST traduce HTTP y JSON:API a llamadas de dominio	JSON:API estandariza recursos, paginación e inclusión de relacionados	ADR-003
Capacidad de interacción	Actualización reactiva de la interfaz sin recarga completa	No aplica directamente al dominio; es decisión del adaptador de interfaz	Vue actualiza el estado en tiempo real tras cada acción	ADR-004
Fiabilidad	Reinicio automático de servicios ante fallo	La separación en adaptadores permite reiniciar uno sin tumbar el dominio	Política restart de Docker Compose reinicia un contenedor caído	ADR-005
Seguridad	Validar la entrada en el borde, separada de las reglas de negocio	Las reglas de negocio se validan en el dominio, no en el borde	Pydantic valida formato y tipos antes de llegar al dominio	ADR-006
Mantenibilidad	Abstraer servicios comunes mediante puertos y adaptadores	El dominio depende de un Protocol, no de una implementación concreta	Protocol de Python y Depends() de FastAPI inyectan el adaptador concreto	ADR-007
Flexibilidad	Contenerizar cada componente para portabilidad entre entornos	Cada adaptador es un módulo independiente y empaquetable por separado	Docker Compose levanta frontend, backend y MongoDB en cualquier máquina	ADR-008
Flexibilidad	Sustituir el adaptador real por un doble de prueba	El puerto define el contrato; cualquier adaptador que lo cumpla es intercambiable	Un adaptador en memoria se inyecta en los tests reemplazando a Depends()	ADR-009
Seguridad de funcionamiento	Prevenir estados inconsistentes que causen daño operativo	La regla de auto-cierre vive en el dominio como invariante	AprobarPostulacion cierra las demás postulaciones en la misma operación	ADR-010

2.3. Registros de Decisión Arquitectónica (ADRs)

A continuación, se documenta para cada táctica de la matriz anterior, el registro de decisión arquitectónica correspondiente:

2.3.1. ADR-001: Casos de Uso Explícitos en la Capa de Dominio

- **Atributo de Calidad ISO/IEC 25010:2023:** Adecuación funcional

- **Contexto:** Si la lógica de postular a un animal o aprobar una postulación se reparte entre el controlador de FastAPI y el modelo de datos, se vuelve fácil implementar el caso de uso de forma incompleta o inconsistente entre distintos endpoints que toquen la misma regla.
- **Decisión:** Cada acción de negocio se implementa como una clase o función de caso de uso explícita en la capa de aplicación del dominio; los endpoints de FastAPI se limitan a invocarla y traducir su resultado a HTTP.
- **Alternativas consideradas:** Se descartó escribir la lógica directamente dentro de la función del endpoint, porque mezcla orquestación HTTP con reglas de negocio y dificulta reutilizar la regla desde otro adaptador.
- **Consecuencias:** Cada regla de negocio tiene un único lugar de verdad, fácil de ubicar y de probar; sin embargo, añade una capa adicional que un sistema CRUD trivial no necesitaría.

2.3.2. ADR-002: Documento Embebido para la Ficha de Animal

- **Atributo de Calidad ISO/IEC 25010:2023:** Eficiencia de desempeño
- **Contexto:** La ficha de un animal, con fotos, ficha de salud y temperamento, varía según la especie y siempre se consulta como una unidad completa en la pantalla de detalle.
- **Decisión:** El animal se modela como un único documento de MongoDB con fotos, ficha de salud y etiquetas embebidos, en vez de colecciones separadas relacionadas por un identificador.
- **Alternativas consideradas:** Se descartaron colecciones separadas con el operador \$lookup para reconstruir la vista, porque agregan consultas adicionales para un dato que siempre se lee y se escribe junto.

- **Consecuencias:** Una sola consulta obtiene todo lo necesario para pintar la ficha del animal; sin embargo, si en el futuro se necesitara una galería global de fotos independiente del animal, este modelo obligaría a recorrer todos los documentos.

2.3.3. ADR-003: JSON:API como Contrato de Integración

- **Atributo de Calidad ISO/IEC 25010:2023:** Compatibilidad
- **Contexto:** El sistema puede ser consumido a futuro por más de un cliente, como la aplicación Vue actual, una eventual app móvil o el panel de un refugio aliado, y cada uno necesita paginación, filtros e inclusión de recursos relacionados de forma consistente.
- **Decisión:** Todos los endpoints REST exponen sus recursos siguiendo la convención JSON:API.
- **Alternativas consideradas:** Se descartó definir un formato JSON propio del equipo, porque cada cliente futuro tendría que aprender un contrato propietario en vez de uno estándar y documentado.
- **Consecuencias:** Cualquier cliente nuevo puede integrarse conociendo únicamente la convención JSON:API; a cambio, se requiere más disciplina en cada endpoint para respetar el formato en cada respuesta, incluidos los errores.

2.3.4. ADR-004: Interfaz Reactiva con Vue en vez de Recarga de Página

- **Atributo de Calidad ISO/IEC 25010:2023:** Capacidad de interacción
- **Contexto:** Cuando un refugio aprueba o rechaza una postulación, o un adoptante postula, el estado visible debe reflejar el cambio de inmediato para que el usuario no piense que su acción falló.
- **Decisión:** El frontend en Vue actualiza el estado local de forma inmediata después de una respuesta exitosa de la API, sin requerir recargar la página completa.

- **Alternativas consideradas:** Se descartó recargar la página tras cada acción, porque ofrece peor experiencia y no aprovecha la reactividad, que es la razón principal para elegir Vue.
- **Consecuencias:** El usuario recibe retroalimentación inmediata; a cambio, es necesario mantener sincronizado el estado local del frontend con el estado real del backend.

2.3.5. ADR-005: Políticas de Reinicio Automático en Docker Compose

- **Atributo de Calidad ISO/IEC 25010:2023:** Fiabilidad
- **Contexto:** En desarrollo y en la demostración de la sustentación, un contenedor, típicamente el de MongoDB, puede caerse por falta de recursos momentánea; sin una política de recuperación, el sistema queda inutilizable hasta un reinicio manual.
- **Decisión:** Se configura la política de reinicio ante fallo en cada servicio del archivo docker-compose.
- **Alternativas consideradas:** Se descartó no configurar ninguna política de reinicio, porque un fallo durante la sustentación detendría toda la presentación sin necesidad.
- **Consecuencias:** Se obtiene recuperación automática ante caídas transitorias; sin embargo, esto no resuelve fallos persistentes, como un error de configuración, en cuyo caso el contenedor entrará en un ciclo de reinicios sin éxito.

2.3.6. ADR-006: Validación de Esquema en el Borde, Separada de las Reglas de Negocio

- **Atributo de Calidad ISO/IEC 25010:2023:** Seguridad
- **Contexto:** Existen dos tipos de validación distintos: de formato, como si un campo es una cadena no vacía, y de negocio, como si el animal sigue disponible. Mezclarlas dificulta reutilizar las reglas de negocio si se agrega otro canal de entrada.

- **Decisión:** Los modelos de Pydantic en FastAPI validan exclusivamente formato y tipos en el borde del sistema; las reglas de negocio se validan dentro de los casos de uso del dominio, lanzando excepciones de dominio propias.
- **Alternativas consideradas:** Se descartó validar todo, formato y negocio, en el modelo de Pydantic, porque acopla las reglas de negocio a FastAPI.
- **Consecuencias:** Las entradas malformadas se rechazan antes de tocar el dominio y las reglas de negocio son reutilizables sin importar el adaptador de entrada; a cambio, se requiere mantener dos capas de validación y un mapeo explícito de excepciones de dominio a códigos HTTP.

2.3.7. ADR-007: Puertos de Repositorio con Adaptador MongoDB

- **Atributo de Calidad ISO/IEC 25010:2023:** Mantenibilidad
- **Contexto:** El dominio necesita persistir y consultar las entidades Animal y Postulación sin acoplarse a un motor de base de datos específico, para no obligar a reescribir casos de uso si cambia la tecnología de persistencia.
- **Decisión:** Se definen los puertos de repositorio como Protocol de Python en el dominio, implementados por un adaptador concreto construido con Motor y Beanie, inyectado mediante Depends() de FastAPI.
- **Alternativas consideradas:** Se descartó acceder a Mongo directamente desde los endpoints, porque acopla el dominio a la infraestructura.
- **Consecuencias:** Cambiar de motor de persistencia en el futuro solo implicaría escribir un nuevo adaptador; a cambio, se añade una capa de indirección que un sistema CRUD simple no necesitaría.

2.3.8. ADR-008: Contenerización de los Tres Servicios con Docker Compose

Atributo de Calidad ISO/IEC 25010:2023: Flexibilidad

Contexto: El equipo está compuesto por tres desarrolladores en máquinas con sistemas operativos distintos, y el sistema tiene piezas con entornos de ejecución distintos: Vue con Node, FastAPI con Python y MongoDB.

Decisión: Cada pieza se empaqueta en su propio contenedor, orquestados con un único archivo de Docker Compose.

Alternativas consideradas: Se descartó instalar Node, Python y MongoDB directamente en cada máquina, porque reintroduce el problema de que algo funcione solo en una máquina y no cumple el requisito obligatorio de contenerización de la actividad.

Consecuencias: Se logra un entorno idéntico para los tres integrantes y para la demostración de la sustentación en cualquier equipo; a cambio, se requiere configurar red y variables de entorno para que los servicios se descubran entre sí.

2.3.9. ADR-009: Adaptador en Memoria para Pruebas del Dominio

- **Atributo de Calidad ISO/IEC 25010:2023: Flexibilidad**
- **Contexto:** Se necesita probar las reglas de negocio, como no postular a un animal no disponible o el auto-cierre de postulaciones, sin depender de una instancia real de MongoDB en cada corrida de pruebas, en un equipo de tres personas trabajando en paralelo con tiempo limitado.
- **Decisión:** Se implementa un adaptador en memoria que cumple el mismo contrato que el adaptador de MongoDB, respaldado por estructuras en memoria, usado exclusivamente en las pruebas unitarias del dominio.
- **Alternativas consideradas:** Se descartó levantar una instancia real de MongoDB en cada prueba, por ser más lenta y agregar una dependencia externa a pruebas que deberían

validar solo lógica de negocio; también se descartó no escribir pruebas unitarias del dominio, porque no aislaría fallos de negocio de fallos de infraestructura.

- **Consecuencias:** Se obtienen pruebas del dominio rápidas y sin dependencias externas, con evidencia concreta de que Hexagonal mejora la capacidad de prueba; a cambio, el adaptador en memoria debe mantenerse sincronizado con el contrato del puerto real.

2.3.10. ADR-010: Auto-Cierre de Postulaciones para Prevenir Doble Adopción

- **Atributo de Calidad ISO/IEC 25010:2023:** Seguridad de funcionamiento
- **Contexto:** Si dos postulaciones pendientes sobre el mismo animal pudieran aprobarse por separado, por ejemplo, por dos operadores del refugio actuando casi al mismo tiempo, el mismo animal podría adjudicarse a dos adoptantes distintos, una inconsistencia que se traduce en daño operativo real.
- **Decisión:** El caso de uso de aprobar una postulación cambia el estado del animal a adoptado y cierra automáticamente cualquier otra postulación pendiente sobre ese mismo animal, como parte de la misma operación de dominio, tratado como un invariante y no como una validación opcional de la interfaz.
- **Alternativas consideradas:** Se descartó dejar que el frontend deshabilite el botón de aprobar en las demás postulaciones, por ser una validación de interfaz y no una garantía real ante dos operadores actuando en paralelo; también se descartó notificar al refugio para un cierre manual, porque deja una ventana de tiempo con un estado inconsistente.
- **Consecuencias:** El sistema garantiza por diseño que un animal nunca queda adoptado por más de una postulación aprobada; a cambio, la operación de aprobar una postulación requiere actualizar varios documentos de forma coordinada.

2.4. Matriz de Atributos de Calidad vs. Estilo Hexagonal

A diferencia de la matriz de tácticas, que documenta decisiones puntuales, esta matriz evalúa el estilo Hexagonal en sí mismo, mostrando tanto lo que soporta como lo que limita:

Tabla 4: Atributos de calidad frente al estilo Hexagonal

Atributo de calidad	Impacto	Cómo lo soporta o limita Hexagonal	Evidencia en el proyecto
Testabilidad	Alto	El dominio y los casos de uso dependen de puertos, no de Mongo ni de FastAPI. Se prueban con adaptadores in-memory o mocks, sin levantar base de datos ni servidor.	Adaptador in-memory del backend y pruebas unitarias del dominio mockeando puertos. En el frontend, el adaptador simulado permite probar todas las pantallas sin backend.
Modificabilidad	Alto	Un cambio de infraestructura queda encerrado en un adaptador; el núcleo no se toca.	Cambiar de Mongo a otra base de datos implicaría escribir otro adaptador que implemente los mismos puertos. Pasar de datos simulados a la API real en la SPA es cambiar una variable de entorno, sin tocar vistas.
Modularidad	Alto	Frontera explícita entre núcleo (dominio, casos de uso, puertos) y periferia (adaptadores). Cada pieza tiene una sola razón para cambiar.	Estructura del backend organizada en dominio, puertos, casos de uso y adaptadores de entrada y salida; los tres desarrolladores trabajaron en paralelo sobre esa frontera.
Analizabilidad	Medio	Las reglas de negocio viven en un solo lugar y no se mezclan con código de HTTP o de base de datos.	La regla de auto-cierre está concentrada en el caso de uso de aprobar postulación y en los métodos del dominio.
Reusabilidad	Medio	Los casos de uso son reutilizables por cualquier adaptador de entrada, como REST, un CLI o tareas programadas.	El mismo caso de uso de postular podría ser invocado por un script de carga masiva sin duplicar validaciones.
Adaptabilidad / Portabilidad	Alto	La tecnología es un detalle reemplazable: el núcleo es Python puro, sin importar FastAPI ni Motor.	Todo corre en contenedores Docker; el núcleo se podría portar a otro framework web cambiando solo el adaptador de entrada.
Interoperabilidad	Medio	Se pueden añadir varios adaptadores de entrada o salida en paralelo para hablar con otros sistemas.	Hoy: REST/JSON:API. Añadir un adaptador de eventos o notificaciones por correo no afectaría al dominio.
Fiabilidad	Medio	Las invariantes se validan en el dominio, no en el controlador ni en el frontend, así que ningún canal de entrada puede saltárselas.	Reglas como no postular a un animal adoptado o el auto-cierre se validan en el núcleo; un índice único parcial en Mongo impide además dos postulaciones pendientes del mismo adoptante al mismo animal.
Seguridad	Medio	El estilo no aporta autenticación por sí mismo, pero facilita aplicarla en el adaptador de entrada sin contaminar el dominio.	Fuera del alcance del taller, sin inicio de sesión. De agregarse, iría como dependencia de FastAPI en el adaptador REST.
Eficiencia de desempeño	Medio	Las capas de indirección entre DTO, entidad y documento agregan mapeos y copias.	Impacto despreciable a la escala del proyecto; optimizaciones específicas de Mongo pueden resolverse dentro del propio adaptador.

Escalabilidad	Bajo	Hexagonal organiza el interior de un servicio; no define cómo se distribuye. Sí facilita partir el sistema después, porque los límites ya están trazados.	Backend sin estado en su propio contenedor, replicable detrás de un proxy.
Disponibilidad	Bajo	No depende del estilo; la aporta la infraestructura.	Verificaciones de salud en los contenedores y dependencias condicionadas a servicios saludables en el archivo de Docker Compose.
Observabilidad	Medio	Registro de eventos y métricas se pueden añadir como decoradores de puertos o middleware en adaptadores, sin tocar reglas de negocio.	Endpoint de salud que verifica la conexión con Mongo.
Usabilidad	Bajo	No depende del estilo; indirectamente ayuda porque errores de dominio con código semántico producen mensajes claros en la interfaz.	Los errores con código y puntero de origen permiten que la SPA marque el campo exacto que falló.
Simplicidad	Medio	Más archivos, interfaces y mapeos que un sistema CRUD en capas; en sistemas pequeños puede ser sobreingeniería.	Riesgo reconocido: el dominio es pequeño, con cuatro entidades; se justifica por la regla de auto-cierre y por el valor didáctico.
Productividad inicial	Medio	Hay que definir puertos y contratos antes de implementar; si el contrato cambia, bloquea a varios integrantes.	Mitigado con el contrato JSON:API cerrado primero y datos simulados en ambos lados del equipo.

Hexagonal gana con claridad en en todas sus subcaracterísticas al ser un estilo pensado para que la lógica de negocio sobreviva a los cambios de tecnología.

A su vez, el impacto es bajo en escalabilidad, disponibilidad y usabilidad, pues estos atributos dependen principalmente del stack y de la infraestructura, no de la forma interna del código, por lo que afirmar que Hexagonal por sí solo hace que el sistema sea escalable es un error común. Por otro lado, en seguridad el impacto es medio ya que el estilo no aporta autenticación por sí mismo, pero facilita aplicarla sin contaminar el dominio.

Sin embargo, donde el estilo tiene un costo es en desempeño, aunque de forma marginal en la práctica, en simplicidad y en productividad inicial porque el costo se paga al principio, al diseñar puertos y escribir mapeos, y se recupera cuando el sistema cambia o crece.

Para este caso, en una plataforma de adopción con reglas de estado no triviales, como la postulación única activa y el auto-cierre al aprobar, priorizar la corrección y la capacidad de

prueba de las reglas sobre el rendimiento bruto es la decisión correcta, y Hexagonal lo permite sin sacrificar ningún atributo crítico para este dominio.

2.5. Matriz de Mercado Laboral vs. Estilo y Stack

Tabla 3: Demanda, salario y proyección de empleo del estilo y el stack

Pieza del Stack	Demanda	Salario junior en Colombia (COP/mes)	Salario general en EE. UU. (USD/año)	Proyección de Empleo
Hexagonal (proxy: Arquitecto de Software)	Ningún índice mide el estilo por nombre. Gasto mundial en la nube pública: US\$723.400 millones en 2025, +21,5% interanual (Gartner, Inc., 2024)	\$2.812.680 COP (Indeed, 2026)	US\$135.980 anuales; mediana de Software Developers (Bureau of Labor Statistics [BLS], 2025); el BLS no distingue “arquitecto” como ocupación aparte	+10% de empleo, 2025-2035 (BLS, 2025)
Python	N.º 1 en TIOBE, con 21,81% en febrero de 2026 (TIOBE Software, 2026); N.º 1 en PYPL, con cerca de 50% de búsquedas de tutoriales (PYPL, s.f.)	\$2.812.680 COP (Indeed, 2026)	US\$135.980 anuales; mediana de Software Developers (BLS, 2025)	+10% de empleo, 2025-2035 (BLS, 2025)
FastAPI	Pasó de 29% a 38% de adopción entre desarrolladores de Python en un año; framework de Python de mayor crecimiento, por encima de Django y Flask (Stack Overflow, 2025)	\$2.812.680 COP (Indeed, 2026)	US\$135.980 anuales; mediana de Software Developers (BLS, 2025); el BLS no reporta por framework	Mayor crecimiento de adopción de cualquier framework de Python en cuatro años (DZone, 2025)
Vue.js	JavaScript sigue siendo el lenguaje más usado, con 66%; Vue es el tercer framework de frontend, tras React y Angular (Stack Overflow, 2025)	\$2.812.680 COP (Indeed, 2026)	US\$92.650 anuales; mediana de Web Developers (BLS, 2025)	+4% de empleo, 2025-2035 (BLS, 2025)
MongoDB	PostgreSQL lidera con 58,2%; Mongo se mantiene entre las bases de datos de código abierto dominantes (JetBrains, 2025)	\$2.812.680 COP (Indeed, 2026)	US\$139.500 anuales; mediana de Database Architects (BLS, 2025)	+9% de empleo, 2025-2035 (BLS, 2025); mercado NoSQL de US\$11.950 a US\$67.450 millones entre 2024 y 2030 (SkyQuest Technology, s.f.)
Docker	+17 puntos porcentuales de adopción interanual, el mayor salto de una sola tecnología en la encuesta (Stack Overflow, 2025)	\$2.812.680 COP (Indeed, 2026)	US\$87.011 anuales; mediana global de DevOps Engineer (Stack Overflow,	Mercado de orquestación de contenedores de US\$1.870 a US\$9.060 millones

			2025); el BLS no tiene categoría propia	entre 2024 y 2030 (Grand View Research, s.f.)
REST / JSON:API	REST es usado por 93% de los equipos de API; JSON:API es una especificación sobre REST, versión 1.1 desde 2022 (Postman, 2025)	\$2.812.680 COP (Indeed, 2026)	US\$135.980 anuales; mediana de Software Developers (BLS, 2025)	+10% de empleo, 2025-2035 (BLS, 2025)

2.6. Matriz de análisis de Principios vs estilo

Esta matriz evalúa cómo el estilo Hexagonal favorece o dificulta cada principio de diseño, con ejemplos tomados del código real del backend. La escala es: cumple; cumple con excepciones conocidas; no cumple. De los doce principios evaluados, seis se cumplen plenamente y seis con excepciones identificadas y justificadas; ninguno se incumple.

Tabla 5: Principios de diseño frente al estilo Hexagonal

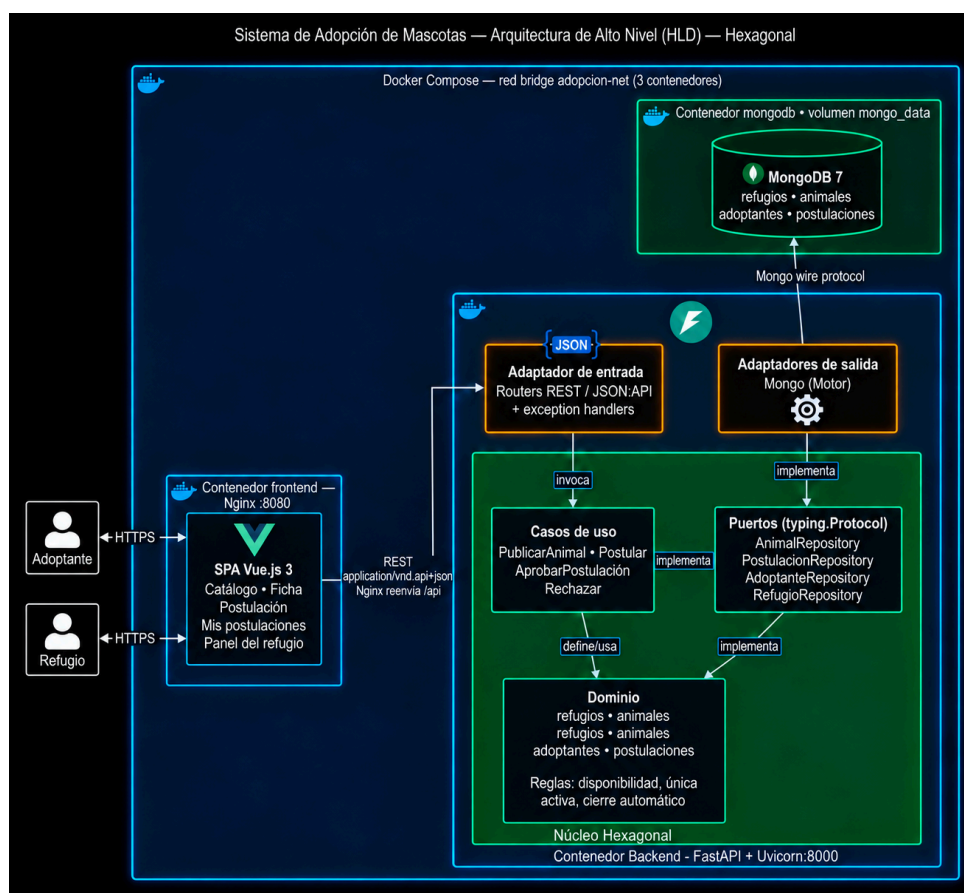
Principio	Cómo lo favorece Hexagonal	Cumplimiento	Ejemplo en el código
S — Responsabilidad única	Separa el núcleo (reglas) de la periferia (HTTP, base de datos); cada adaptador tiene una sola razón para cambiar.	Cumple	Un caso de uso por clase: Postular, AprobarPostulacion y RechazarPostulacion. El módulo jsonapi solo traduce y el módulo errors solo mapea errores.
O — Abierto/cerrado	Una tecnología nueva es un adaptador nuevo, sin tocar el núcleo.	Cumple	Una excepción de dominio nueva hereda el código HTTP de su padre sin modificar el manejador de errores.
L — Sustitución de Liskov	Todo adaptador que cumple el puerto es intercambiable.	Cumple	Los mismos casos de uso corren con el adaptador en memoria, con mocks y con MongoDB sin cambiar una línea.
I — Segregación de interfaces	Un puerto por necesidad del núcleo, no un repositorio genérico.	Cumple	Cuatro puertos pequeños; RefugioRepository solo expone obtener y listar.
D — Inversión de dependencias	Es el corazón del estilo: el núcleo define las	Cumple	Los casos de uso reciben AnimalRepository (un

	interfaces y la infraestructura las implementa.		Protocol); la implementación concreta la decide el módulo dependencies con Depends().
KISS	Neutral: el estilo añade capas y la simplicidad debe cuidarse dentro de cada una.	Con excepciones	Aprobar o rechazar se decide en una línea; el costo es tener más archivos que un CRUD.
DRY	Las reglas viven en un solo lugar, el dominio, y no en cada canal de entrada.	Con excepciones	Una sola tabla de excepciones a códigos HTTP (MAPEO_DOMINIO); la validación de formato se repite a propósito en el frontend.
YAGNI	Los puertos solo declaran lo que un caso de uso usa hoy.	Cumple	Sin login, sin ORM, sin repositorio genérico ni transacciones, porque el taller no los requiere.
PoLA — Menor sorpresa	Los contratos explícitos (puertos y JSON:API) hacen predecible el comportamiento.	Con excepciones	Los repositorios devuelven copias, como una base de datos real. Sorpresa conocida: registrar un adoptante con un correo existente responde 200.
Ley de Demeter	El caso de uso habla con puertos y entidades, no con sus detalles internos.	Con excepciones	La entidad valida si el animal es postulable en lugar de revisar su estado desde fuera. Violación conocida: el router recorre relaciones anidadas del cuerpo JSON:API.
Composición sobre herencia	Los adaptadores se componen en los casos de uso; no se heredan.	Con excepciones	Los casos de uso reciben sus puertos por constructor. Excepción: una clase base comparte los repositorios por herencia.
STUPID (anti-principios)	Hexagonal ataca sobre todo el acoplamiento fuerte y la baja testabilidad.	Con excepciones	Se evitan 4 de 6; quedan dos singletons controlados (el cliente de MongoDB y la fábrica de repositorios).

III. DISEÑO DE EJEMPLO PRÁCTICO

3.1. Diagrama de Alto Nivel (HLD)

Figura 1: Diagrama de alto nivel del sistema

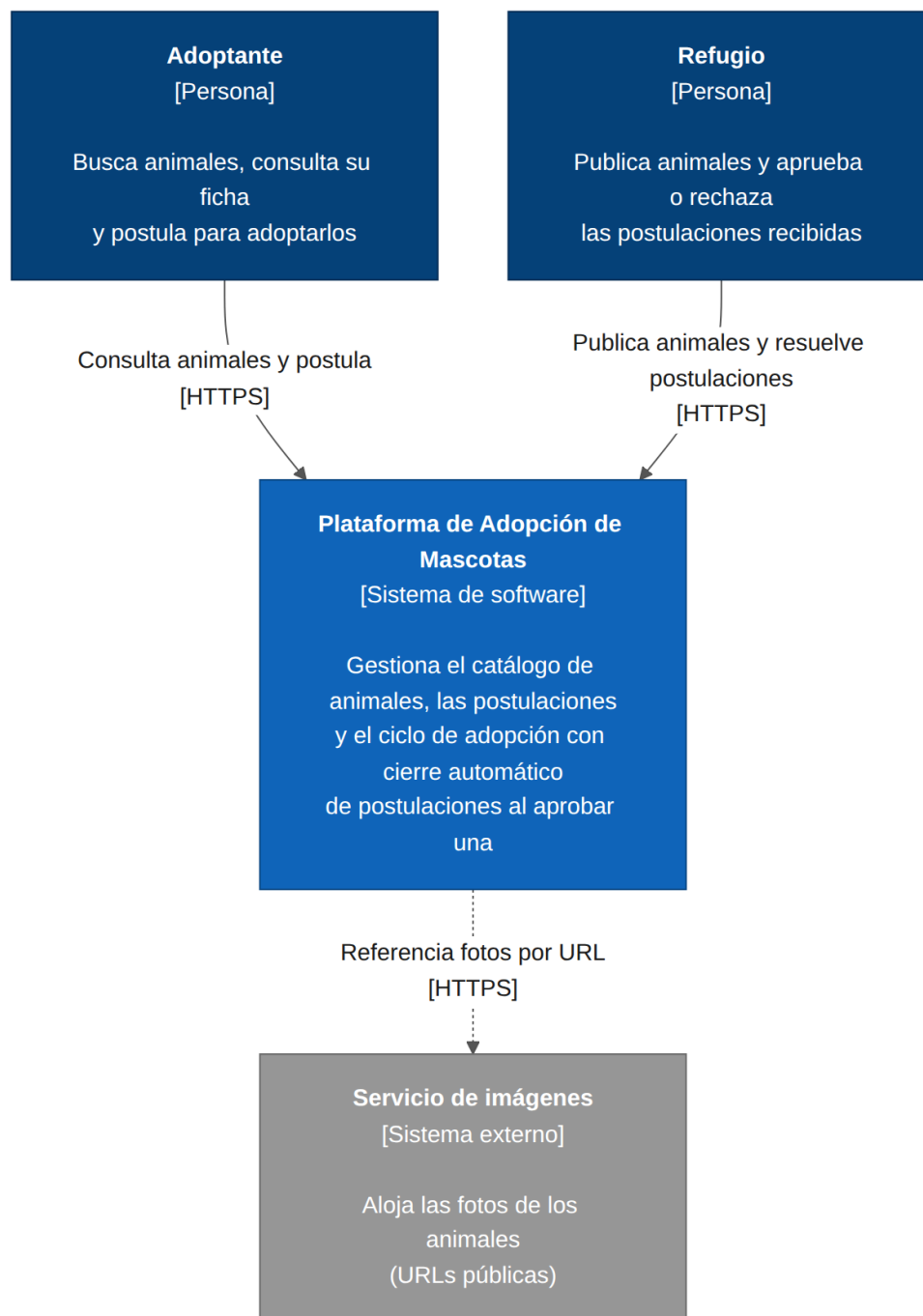


El diagrama de alto nivel resume la solución completa desplegada con Docker Compose. El adoptante y el refugio usan la SPA en Vue.js servida por Nginx, que reenvía las peticiones /api al backend en FastAPI mediante REST con el formato JSON:API. Dentro del backend se distinguen tres zonas: el adaptador de entrada (routers REST y manejo de errores), el núcleo hexagonal (casos de uso, entidades del dominio con sus reglas y puertos definidos como Protocol) y los adaptadores de salida hacia MongoDB o hacia memoria para las pruebas. La

relación entre los adaptadores de salida y los puertos evidencia la inversión de dependencias: la infraestructura implementa los contratos que define el núcleo, y nunca al revés.

3.2. Diagrama de Contexto (Nivel 1)

Figura 2: Diagrama de contexto (Nivel 1)

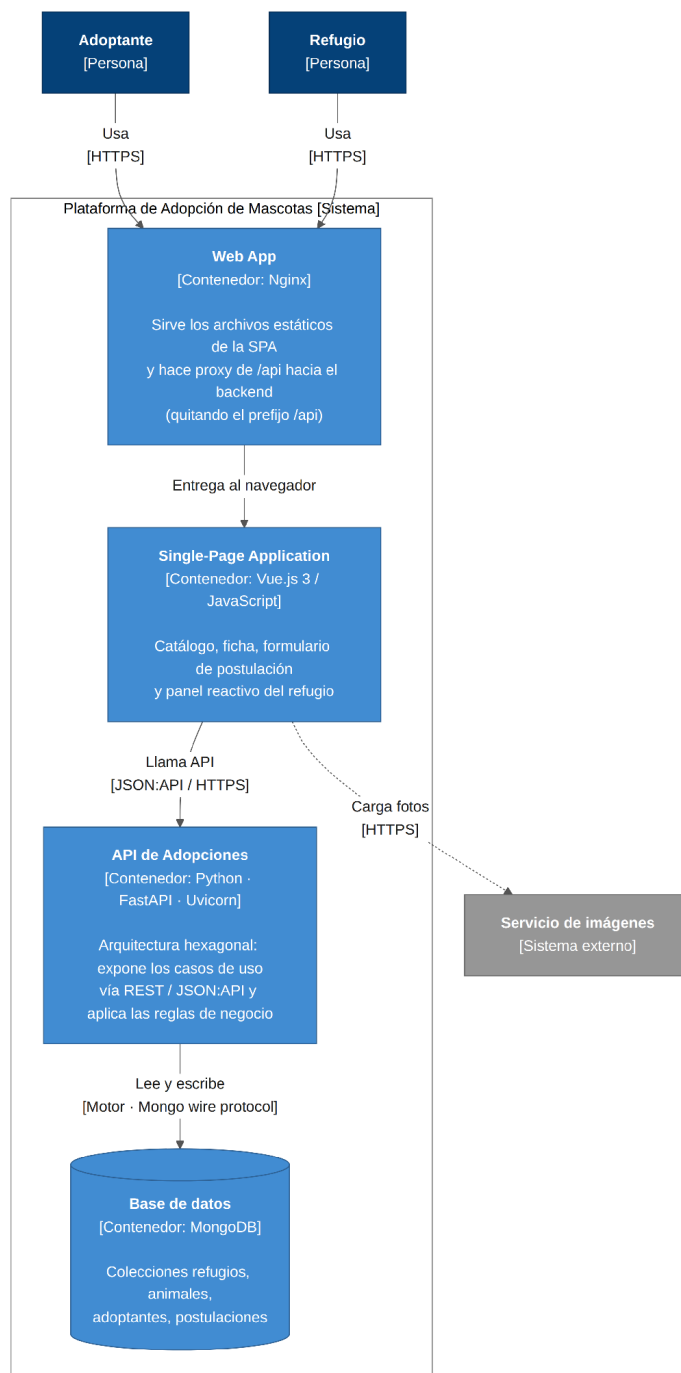


El diagrama de contexto muestra la plataforma como una sola caja y define su frontera. Los dos actores humanos son el adoptante, que consulta animales y postula para adoptarlos, y el refugio, que publica animales y aprueba o rechaza las postulaciones recibidas; ambos interactúan por HTTPS. El único sistema externo es el servicio de imágenes, pues la plataforma solo guarda la

URL pública de cada foto. En este nivel ya aparece la regla central del negocio: el cierre automático de las demás postulaciones cuando se aprueba una.

3.3. Diagrama de Contenedores (Nivel 2)

Figura 3: Diagrama de contenedores (Nivel 2)

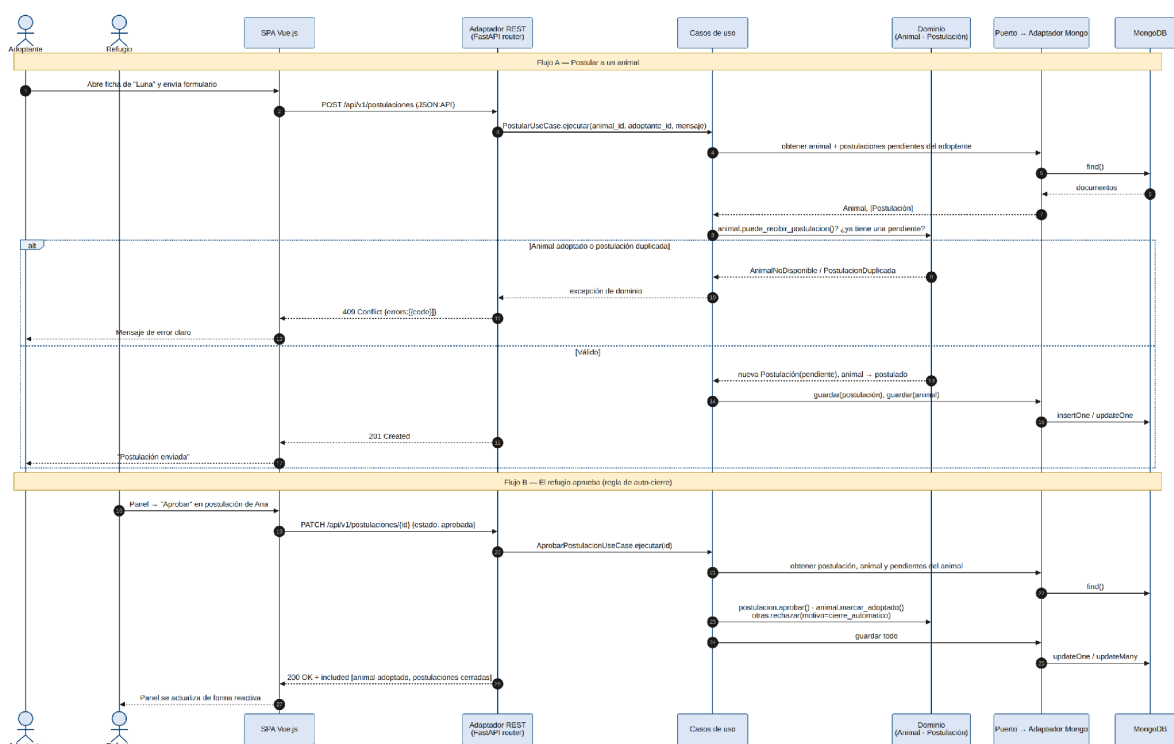


El diagrama de contenedores abre la plataforma en sus cuatro piezas desplegables. Nginx entrega la aplicación de una sola página al navegador y hace de proxy de /api hacia el backend, quitando ese prefijo. La SPA en Vue muestra el catálogo, la ficha, el formulario de postulación y el panel del refugio, y se comunica con la API de adopciones usando JSON:API. La API, construida con

FastAPI y Uvicorn, es donde vive la arquitectura hexagonal: expone los casos de uso y aplica las reglas de negocio. Finalmente, MongoDB guarda las colecciones de refugios, animales, adoptantes y postulaciones, y el backend accede a ella con el driver asíncrono Motor.

3.4. Diagrama Dinámico

Figura 4: Flujo dinámico principal: postular y aprobar una adopción

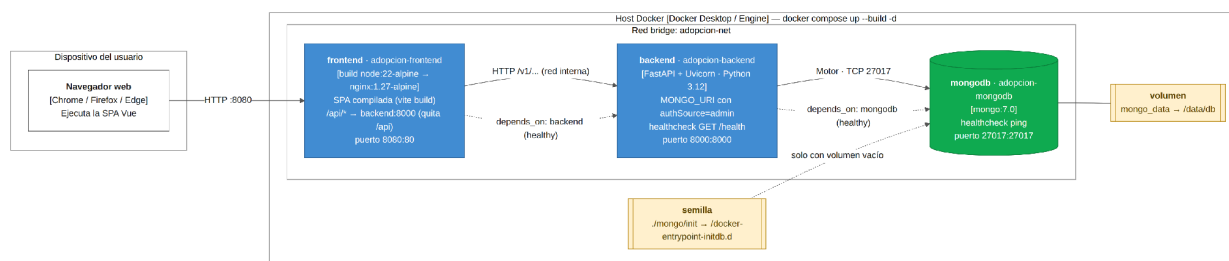


El diagrama dinámico recorre los dos flujos que sostienen el caso. En el flujo A, el adoptante envía su postulación; el adaptador REST invoca el caso de uso Postular, que carga el animal y las postulaciones pendientes a través del puerto de repositorio y pregunta al dominio si el animal puede recibirla. Si el animal no está disponible o ya existe una postulación pendiente del mismo adoptante, el dominio lanza una excepción y la API responde 409 con un error en formato JSON:API; si es válida, se guarda y responde 201. En el flujo B, el refugio aprueba una postulación con un PATCH; el caso de uso AprobarPostulacion marca el animal como adoptado

y rechaza en la misma operación las demás postulaciones pendientes con motivo de cierre automático, y el panel del refugio se actualiza sin recargar la página.

3.5. Diagrama de Despliegue

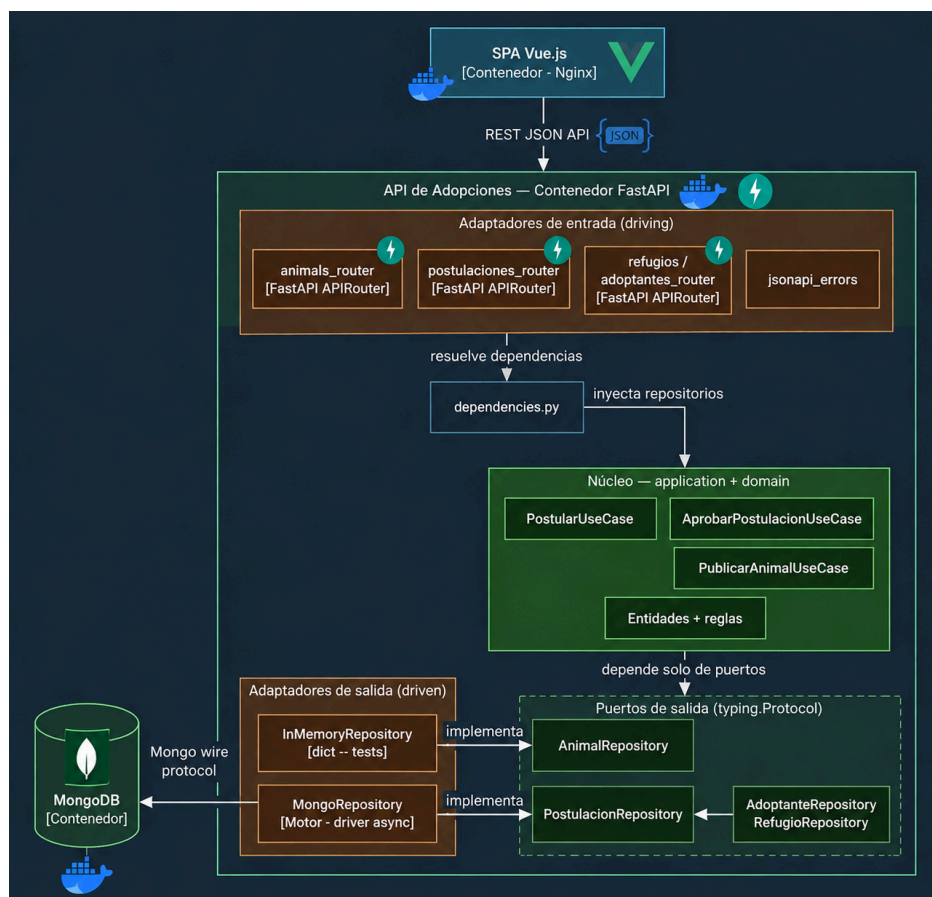
Figura 5: Diagrama de despliegue



El diagrama de despliegue muestra cómo se ejecuta el sistema con un solo comando, `docker compose up --build`. Los tres contenedores comparten la red bridge `adopcion-net`: el frontend (build de Node y servidor Nginx, publicado en el puerto 8080), el backend (FastAPI con Uvicorn, puerto 8000) y MongoDB 7 (puerto 27017). El navegador del usuario solo habla con Nginx, que reenvía `/api` al backend por la red interna. El orden de arranque lo controlan los healthchecks: el backend espera a que MongoDB responda y el frontend espera al backend. Los datos persisten en el volumen `mongo_data`, y la semilla inicial solo se carga cuando ese volumen está vacío.

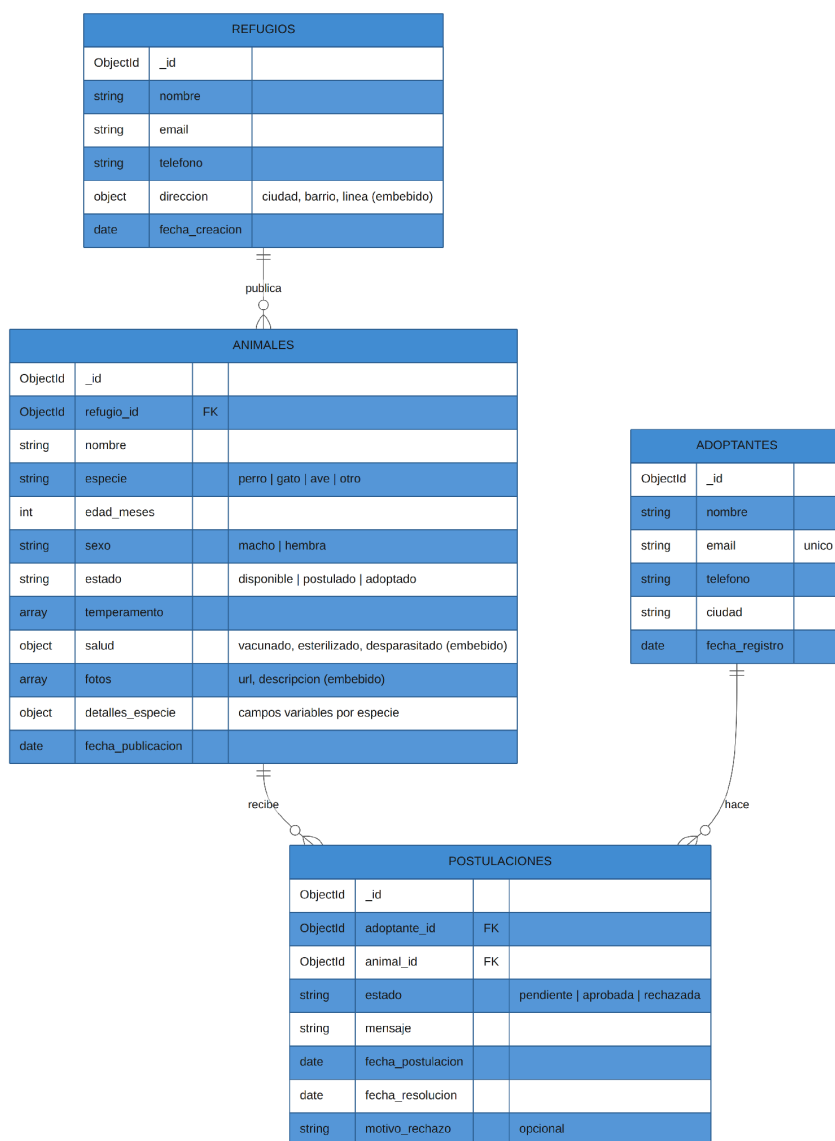
3.6. Diagrama de Componentes (Nivel 3)

Figura 6: Diagrama de componentes (Nivel 3)



El diagrama de componentes entra en el contenedor de la API y muestra el estilo hexagonal en el código real. Los adaptadores de entrada son los routers de FastAPI (animals, postulaciones, refugios y adoptantes) y el manejador jsonapi_errors, que traduce las excepciones del dominio a códigos HTTP. El archivo dependencies.py actúa como raíz de composición: es el único lugar que decide qué adaptador concreto se inyecta. El núcleo reúne los casos de uso (Postular, AprobarPostulacion y PublicarAnimal) y las entidades con sus reglas, y depende solo de los puertos de salida definidos con typing.Protocol. Del otro lado, los repositorios en memoria y los de MongoDB implementan esos mismos puertos, por lo que se pueden intercambiar sin modificar el núcleo.

3.7. Modelo de Datos

Figura 7: Modelo de datos del sistema

El modelo de datos en MongoDB tiene cuatro colecciones: refugios, animales, adoptantes y postulaciones. Siguiendo el ADR-002, el documento de cada animal embebe lo que siempre se consulta junto con él: la salud (vacunado, esterilizado, desparasitado), el arreglo de fotos y los detalles propios de cada especie, que varían entre perros, gatos y aves sin obligar a crear tablas adicionales. Las relaciones entre agregados se guardan como referencias por identificador: el animal apunta a su refugio y cada postulación apunta a un animal y a un adoptante. Los estados

están acotados (disponible, postulado o adoptado para el animal; pendiente, aprobada o rechazada para la postulación) y la postulación registra la fecha de resolución y el motivo de rechazo, que permite distinguir un rechazo manual del cierre automático que ocurre cuando se aprueba otra postulación para el mismo animal. El correo del adoptante es único.

IV. LECCIONES APRENDIDAS

- Python es, hoy, la apuesta más segura del stack: lidera TIOBE y PYPL simultáneamente, algo sin precedentes en la historia de ambos índices (PYPL, s.f.; TIOBE Software, 2026).
- FastAPI es la pieza de mayor momentum: pasó de ser una opción moderna a ser el framework de Python más usado en un año, con adopción empresarial real en Netflix, Microsoft y Uber (DZone, 2025).
- Docker ya no es una decisión diferenciadora, sino una expectativa básica del mercado, con el mayor salto de adopción de cualquier tecnología registrado en la encuesta de 2025 (Stack Overflow, 2025).
- MongoDB y Vue son piezas maduras y estables, sin el crecimiento de las anteriores, pero con demanda sostenida y sin señales de declive.
- El equipo construye con una combinación donde tres de cuatro piezas, Python, FastAPI y Docker, están en su punto más alto o de mayor crecimiento histórico, un argumento de mercado laboral sólido para la elección del stack, más allá de lo técnico.
- El esquema flexible de MongoDB modeló la ficha de “Animal” según cada especie sin tablas vacías ni JOINS, pero trasladó la disciplina de validación del motor al adaptador y a las pruebas.

- El contrato JSON:API permitió construir y probar el frontend antes de que existiera el backend real, y Docker unificó el entorno de tres desarrolladores en distintos sistemas operativos.
- Hexagonal ganó impacto Alto en Testabilidad, Modificabilidad y Modularidad, a costa de un impacto Medio reconocido en Simplicidad y Productividad inicial.
- El mismo mecanismo de puertos intercambiables resuelve tanto la portabilidad vía Docker como la sustitución por un adaptador en memoria en pruebas, lo que explica por qué el estándar unió ambas bajo Flexibilidad.
- Ningún índice lo mide por nombre entre Hexagonal y el mercado, pero que el BLS proyecte más crecimiento para *Database Architects* que para *Database Administrators* es una señal indirecta de que el mercado valora el diseño desacoplado.

Además de la lectura del mercado, la construcción del ejemplo práctico dejó lecciones propias del equipo:

Cerrar el contrato antes de implementar permite trabajar en paralelo. Definir el contrato JSON:API al inicio del segundo sprint hizo que backend y frontend avanzaran al mismo tiempo sin bloquearse; la integración final se redujo a ajustes menores, porque ambos lados ya hablaban el mismo formato de recursos y de errores.

Hexagonal paga su costo en las pruebas. Ubicar las reglas de negocio en casos de uso y probarlas con repositorios en memoria permitió validar el dominio completo, incluido el cierre automático de postulaciones, sin levantar MongoDB ni un servidor web. El precio es más código inicial (puertos, adaptadores y mapeadores), que en un CRUD simple no se justificaría.

El mismo estilo sirve en el frontend. Tratar el acceso a la API como un puerto con dos adaptadores, uno real y uno simulado, dejó construir y probar toda la interfaz antes de que existiera el backend; conectar la API real no modificó ninguna vista. La condición es que el adaptador simulado respete las mismas reglas de negocio que el real, porque de lo contrario oculta errores que aparecen tarde.

La persistencia real y los datos semilla deben integrarse temprano. El adaptador en memoria acelera el desarrollo, pero el sistema solo se valida de extremo a extremo cuando Docker Compose levanta el adaptador de MongoDB con datos iniciales; dejar ese paso para el final concentra el riesgo en la semana de la entrega.

Las decisiones documentadas ahorran discusiones. Registrar cada decisión como ADR y cada tarea en Jira con ramas por funcionalidad permitió justificar el diseño con evidencia y retomar el trabajo de otro integrante sin depender de explicaciones verbales.

Referencias

arc42 Quality Model. (s.f.-a). *Update on ISO 25010, version 2023*. Recuperado el 28 de septiembre de 2026, de <https://quality.arc42.org/articles/iso-25010-update-2023>

arc42 Quality Model. (s.f.-b). *ISO/IEC 25010 — Systems and software quality*. Recuperado el 28 de septiembre de 2026, de <https://quality.arc42.org/standards/iso-25010>

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.^a ed.). Addison-Wesley.

Bureau of Labor Statistics. (2025). *Database administrators and architects*. Occupational Outlook Handbook, U.S. Department of Labor.

<https://www.bls.gov/ooh/computer-and-information-technology/database-administrators.htm>

Bureau of Labor Statistics. (2025). *Software developers, quality assurance analysts, and testers*.

Occupational Outlook Handbook, U.S. Department of Labor.

<https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm>

Bureau of Labor Statistics. (2025). *Web developers and digital designers*. Occupational Outlook Handbook, U.S. Department of Labor.

<https://www.bls.gov/ooh/computer-and-information-technology/web-developers.htm>

Cockburn, A. (2005). *Hexagonal architecture (ports and adapters)*.

<https://alistair.cockburn.us/hexagonal-architecture/>

Drupal. (2019). *JSON:API lands in Drupal core*.

<https://www.drupal.org/blog/jsonapi-lands-in-drupal-core>

DZone. (2025). *How FastAPI became Python's fastest-growing framework*.

<https://dzone.com/articles/how-fastapi-became-pythons-fastest-growing-framework>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].

Gartner, Inc. (2024, 19 de noviembre). *Gartner forecasts worldwide public cloud end-user spending to total \$723 billion in 2025* [comunicado de prensa].

<https://www.gartner.com/en/newsroom/press-releases/2024-11-19-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-total-723-billion-dollars-in-2025>

GitLab. (s.f.). *Frontend development guidelines*. Recuperado el 28 de septiembre de 2026, de

https://docs.gitlab.com/development/fe_guide

- Grand View Research. (s.f.). *Container orchestration market size, share & trends analysis report*. Recuperado el 28 de septiembre de 2026, de <https://www.grandviewresearch.com/industry-analysis/container-orchestration-market-report>
- Indeed. (2026). *Sueldo de desarrollador/a junior en Colombia*. Recuperado el 28 de septiembre de 2026, de <https://co.indeed.com/career/desarrollador-junior/salaries>
- InfoQ. (2013, 29 de octubre). *dotCloud renamed Docker*. <https://www.infoq.com/news/2013/10/dotcloud-renamed-docker/>
- International Organization for Standardization. (2023). *Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — Product quality model* (ISO/IEC 25010:2023).
- JetBrains. (2025, octubre). *The state of developer ecosystem 2025*. <https://blog.jetbrains.com/research/2025/10/state-of-developer-ecosystem-2025/>
- JSON:API. (s.f.). *About*. Recuperado el 28 de septiembre de 2026, de <https://jsonapi.org/about/>
- JSON:API. (2022). *JSON:API — Specification v1.1*. <https://jsonapi.org/format/1.1/>
- MongoDB, Inc. (s.f.). *Our story*. Recuperado el 28 de septiembre de 2026, de <https://www.mongodb.com/company/our-story>
- Monterail. (2026). *Top companies using Vue.js in 2026*. <https://www.monterail.com/blog/top-companies-using-vue-js>
- Postman. (2025). *2025 state of the API report*. <https://www.postman.com/state-of-api/2025/>
- PYPL. (s.f.). *PopularitY of Programming Language Index*. Recuperado el 28 de septiembre de 2026, de <https://pypl.github.io/>

RepoJournal. (2026, 18 de julio). *Vue 3.6 RC lands with Vapor Mode complete*.

<https://repojournal.com/showcase/vuejs/2026-07-18>

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture*. O'Reilly Media.

SkyQuest Technology. (s.f.). *NoSQL market size, share and growth analysis*. Recuperado el 28

de septiembre de 2026, de <https://www.skyquestt.com/report/nosql-market>

Stack Overflow. (2025). *2025 Stack Overflow developer survey*.

<https://survey.stackoverflow.co/2025/>

TIOBE Software. (2026). *TIOBE index for February 2026*. <https://www.tiobe.com/tiobe-index/>

tkrotoff. (s.f.). *Stack Overflow 2025 and State of JS 2025 survey compilation* [Gist]. GitHub.

<https://gist.github.com/tkrotoff/b1caa4c3a185629299ec234d2314e190>

Vue School. (2025). *Vue.js 2025 in review and a peek into 2026*.

<https://vueschool.io/articles/news/vue-js-2025-in-review-and-a-peek-into-2026/>